

OBJECT-ORIENTED PROGRAMMING WITH JAVA

Complete Practical Tutorial

From Basics to Advanced Concepts

Course Topics:

Introduction to OOP Concepts

Classes and Objects

Constructors and Methods

Encapsulation and Data Hiding

Inheritance (Single, Multilevel, Hierarchical)

Polymorphism (Method Overloading & Overriding)

Abstraction (Abstract Classes & Interfaces)

Exception Handling

Collections Framework

File I/O Operations

Complete Real-World Projects

Best Practices and Design Patterns

TABLE OF CONTENTS

1. MODULE 1: Introduction to OOP
2. MODULE 2: Classes and Objects
3. MODULE 3: Constructors
4. MODULE 4: Methods and Method Overloading
5. MODULE 5: Encapsulation
6. MODULE 6: Inheritance - Single Inheritance
7. MODULE 7: Inheritance - Multilevel and Hierarchical
8. MODULE 8: Method Overriding and super Keyword
9. MODULE 9: Polymorphism
10. MODULE 10: Abstract Classes
11. MODULE 11: Interfaces
12. MODULE 12: Exception Handling
13. MODULE 13: Collections Framework - ArrayList
14. MODULE 14: Collections Framework - HashMap
15. MODULE 15: File I/O Operations
16. MODULE 16: Complete Project - Student Management System
17. MODULE 17: Complete Project - Bank Account System
18. MODULE 18: Complete Project - Library Management System
19. MODULE 19: Best Practices and Design Patterns
20. MODULE 20: Advanced OOP Concepts

MODULE 1: INTRODUCTION TO OOP

1.1 What is Object-Oriented Programming?

Object-Oriented Programming (OOP) is a programming paradigm based on the concept of "objects".

Key Principles:

- Organizes code around objects rather than functions
- Objects contain data (attributes) and code (methods)
- Promotes code reusability and modularity
- Makes complex programs easier to develop and maintain
- Models real-world entities and relationships

1.2 Four Pillars of OOP

1. ENCAPSULATION

- Bundling data and methods together
- Hiding internal details (data hiding)
- Using access modifiers (private, public, protected)

2. INHERITANCE

- Creating new classes from existing ones
- Child class inherits properties from parent class
- Promotes code reusability

3. POLYMORPHISM

- "Many forms" - same method, different implementations
- Method overloading (compile-time polymorphism)
- Method overriding (runtime polymorphism)

4. ABSTRACTION

- Hiding complex implementation details
- Showing only essential features
- Using abstract classes and interfaces

1.3 Real-World Analogy

Think of a Car:

- Class: Car blueprint/design
- Object: Your actual car (Toyota, Honda, etc.)
- Attributes: Color, model, speed, fuel level
- Methods: start(), stop(), accelerate(), brake()
- Encapsulation: You don't need to know how the engine works
- Inheritance: SportsCar extends Car
- Polymorphism: Different cars accelerate differently
- Abstraction: Steering wheel hides complex mechanisms

MODULE 2: CLASSES AND OBJECTS

2.1 Understanding Classes

A class is a blueprint or template for creating objects.

Class contains:

- Attributes (variables/fields) - represent state
- Methods (functions) - represent behavior

2.2 Creating Your First Class

```
// Student.java
public class Student {
    // Attributes (instance variables)
    String name;
    int age;
    String course;
    double gpa;

    // Method to display student information
    public void displayInfo() {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
        System.out.println("Course: " + course);
        System.out.println("GPA: " + gpa);
    }

    // Method to check if student is on honors list
    public boolean isHonors() {
        return gpa >= 3.5;
    }
}
```

2.3 Creating Objects

An object is an instance of a class.

```
// Main.java
public class Main {
    public static void main(String[] args) {
        // Creating object (instantiation)
        Student student1 = new Student();

        // Setting attribute values
        student1.name = "John Kamau";
        student1.age = 20;
    }
}
```

```

        student1.course = "Computer Science";
        student1.gpa = 3.8;

        // Calling methods
        student1.displayInfo();

        if (student1.isHonors()) {
            System.out.println(student1.name + " is on the honors
list!");
        }

        // Creating another object
        Student student2 = new Student();
        student2.name = "Mary Wanjiru";
        student2.age = 21;
        student2.course = "Information Technology";
        student2.gpa = 3.2;

        student2.displayInfo();
    }
}

```

2.4 Understanding the Code

- **class Student** - Declares a new class named Student
- **String name** - Instance variable (attribute) of type String
- **public void displayInfo()** - Method with no return value
- **public boolean isHonors()** - Method returning boolean value
- **Student student1 = new Student()** - Creates new Student object
- **student1.name = "John"** - Accessing object's attribute using dot operator
- **student1.displayInfo()** - Calling object's method

2.5 Practice Exercise

Exercise 1: Create a Book class

- Attributes: title, author, isbn, price
- Methods: displayDetails(), applyDiscount(double percentage)
- Create 3 different book objects and test all methods

MODULE 3: CONSTRUCTORS

3.1 What is a Constructor?

A constructor is a special method that:

- Has the same name as the class
- Has no return type (not even void)
- Is automatically called when object is created
- Used to initialize object attributes
- Can be overloaded (multiple constructors)

3.2 Default Constructor

```
public class Student {
    String name;
    int age;
    String course;
    double gpa;

    // Default Constructor (no parameters)
    public Student() {
        name = "Unknown";
        age = 0;
        course = "Not Assigned";
        gpa = 0.0;
        System.out.println("Default constructor called!");
    }

    public void displayInfo() {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
        System.out.println("Course: " + course);
        System.out.println("GPA: " + gpa);
    }
}

// Usage
public class Main {
    public static void main(String[] args) {
        Student s1 = new Student(); // Constructor called automatically
        s1.displayInfo();
    }
}
```

3.3 Parameterized Constructor

```
public class Student {
    String name;
    int age;
```

```

String course;
double gpa;

// Parameterized Constructor
public Student(String studentName, int studentAge, String
studentCourse, double studentGpa) {
    name = studentName;
    age = studentAge;
    course = studentCourse;
    gpa = studentGpa;
}

public void displayInfo() {
    System.out.println("Name: " + name);
    System.out.println("Age: " + age);
    System.out.println("Course: " + course);
    System.out.println("GPA: " + gpa);
}
}

// Usage
public class Main {
    public static void main(String[] args) {
        // Creating object with constructor
        Student s1 = new Student("Alice Muthoni", 19, "CS", 3.7);
        s1.displayInfo();

        Student s2 = new Student("Bob Ochieng", 22, "IT", 3.4);
        s2.displayInfo();
    }
}

```

3.4 Constructor Overloading

Having multiple constructors with different parameters:

```

public class Student {
    String name;
    int age;
    String course;
    double gpa;

    // Constructor 1: No parameters
    public Student() {
        name = "Unknown";
        age = 0;
        course = "Not Assigned";
        gpa = 0.0;
    }
}

```

```

// Constructor 2: Name and age only
public Student(String name, int age) {
    this.name = name;
    this.age = age;
    this.course = "Not Assigned";
    this.gpa = 0.0;
}

// Constructor 3: All parameters
public Student(String name, int age, String course, double gpa) {
    this.name = name;
    this.age = age;
    this.course = course;
    this.gpa = gpa;
}

public void displayInfo() {
    System.out.println("Name: " + name + ", Age: " + age +
        ", Course: " + course + ", GPA: " + gpa);
}
}

// Usage
public class Main {
    public static void main(String[] args) {
        Student s1 = new Student();
        Student s2 = new Student("Jane Akinyi", 20);
        Student s3 = new Student("Tom Kipchoge", 21, "Engineering",
3.6);

        s1.displayInfo();
        s2.displayInfo();
        s3.displayInfo();
    }
}

```

3.5 The "this" Keyword

The "this" keyword refers to the current object.

Uses of "this":

- Distinguish between instance variables and parameters with same name
- Call one constructor from another
- Pass current object as parameter
- Return current object from a method

```
public class Student {
    String name;
    int age;

    // Using 'this' to distinguish between parameter and instance
variable
    public Student(String name, int age) {
        this.name = name;    // this.name refers to instance variable
        this.age = age;      // name refers to parameter
    }
}
```

MODULE 4: METHODS AND METHOD OVERLOADING

4.1 Types of Methods

1. Methods with no parameters and no return value:

```
public void greet() {  
    System.out.println("Hello, welcome to Java!");  
}
```

2. Methods with parameters but no return value:

```
public void greet(String name) {  
    System.out.println("Hello, " + name + "!");  
}
```

3. Methods with no parameters but return value:

```
public double getGpa() {  
    return gpa;  
}
```

4. Methods with parameters and return value:

```
public double calculateAverage(double score1, double score2, double  
score3) {  
    return (score1 + score2 + score3) / 3.0;  
}
```

4.2 Complete Example with Various Methods

```
public class Calculator {  
  
    // Method 1: Addition with return value  
    public int add(int a, int b) {  
        return a + b;  
    }  
  
    // Method 2: Subtraction with return value  
    public int subtract(int a, int b) {  
        return a - b;  
    }  
  
    // Method 3: Display result (no return)  
    public void displayResult(int result) {  
        System.out.println("Result: " + result);  
    }  
}
```

```

// Method 4: Check if number is even
public boolean isEven(int number) {
    return number % 2 == 0;
}

// Method 5: Calculate percentage
public double calculatePercentage(double obtained, double total) {
    return (obtained / total) * 100;
}
}

// Usage
public class Main {
    public static void main(String[] args) {
        Calculator calc = new Calculator();

        int sum = calc.add(10, 20);
        calc.displayResult(sum);

        int difference = calc.subtract(50, 30);
        calc.displayResult(difference);

        if (calc.isEven(sum)) {
            System.out.println(sum + " is even");
        }

        double percentage = calc.calculatePercentage(450, 500);
        System.out.println("Percentage: " + percentage + "%");
    }
}

```

4.3 Method Overloading

Method overloading allows multiple methods with:

- Same name
- Different parameters (number, type, or order)

```

public class Calculator {

    // Method 1: Add two integers
    public int add(int a, int b) {
        System.out.println("Adding two integers");
        return a + b;
    }

    // Method 2: Add three integers (different number of parameters)
    public int add(int a, int b, int c) {
        System.out.println("Adding three integers");
        return a + b + c;
    }

    // Method 3: Add two doubles (different parameter type)

```

```

    public double add(double a, double b) {
        System.out.println("Adding two doubles");
        return a + b;
    }

    // Method 4: Add integer and double (different parameter order)
    public double add(int a, double b) {
        System.out.println("Adding int and double");
        return a + b;
    }
}

// Usage
public class Main {
    public static void main(String[] args) {
        Calculator calc = new Calculator();

        System.out.println(calc.add(10, 20));           // Calls method 1
        System.out.println(calc.add(10, 20, 30));       // Calls method 2
        System.out.println(calc.add(10.5, 20.3));       // Calls method 3
        System.out.println(calc.add(10, 20.5));         // Calls method 4
    }
}

```

4.4 Practice Exercise

Exercise 2: Create an Area Calculator class

Create overloaded methods to calculate area of:

- Circle: area(double radius)
- Rectangle: area(double length, double width)
- Triangle: area(double base, double height, String shape)
- Test all methods in main()

MODULE 5: ENCAPSULATION

5.1 What is Encapsulation?

Encapsulation is the bundling of data and methods that operate on that data within a single unit (class).

Key Concepts:

- Data hiding - making variables private
- Providing public getter and setter methods
- Controlling access to class members
- Protecting data from unauthorized access

5.2 Access Modifiers

- **private** - Accessible only within the same class
- **public** - Accessible from anywhere
- **protected** - Accessible within package and subclasses
- **default (no modifier)** - Accessible within the same package

5.3 Implementing Encapsulation

```
public class BankAccount {
    // Private attributes (data hiding)
    private String accountNumber;
    private String accountHolder;
    private double balance;

    // Constructor
    public BankAccount(String accountNumber, String accountHolder, double
initialBalance) {
        this.accountNumber = accountNumber;
        this.accountHolder = accountHolder;
        this.balance = initialBalance;
    }

    // Getter methods (read-only access)
    public String getAccountNumber() {
        return accountNumber;
    }

    public String getAccountHolder() {
        return accountHolder;
    }

    public double getBalance() {
```

```

        return balance;
    }

    // Setter method with validation
    public void setAccountHolder(String accountHolder) {
        if (accountHolder != null && !accountHolder.isEmpty()) {
            this.accountHolder = accountHolder;
        } else {
            System.out.println("Invalid account holder name");
        }
    }

    // Business methods
    public void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
            System.out.println("Deposited: KSh " + amount);
            System.out.println("New balance: KSh " + balance);
        } else {
            System.out.println("Invalid deposit amount");
        }
    }

    public void withdraw(double amount) {
        if (amount > 0 && amount <= balance) {
            balance -= amount;
            System.out.println("Withdrawn: KSh " + amount);
            System.out.println("New balance: KSh " + balance);
        } else if (amount > balance) {
            System.out.println("Insufficient funds");
        } else {
            System.out.println("Invalid withdrawal amount");
        }
    }

    public void displayAccountInfo() {
        System.out.println("Account Number: " + accountNumber);
        System.out.println("Account Holder: " + accountHolder);
        System.out.println("Balance: KSh " + balance);
    }
}

// Usage
public class Main {
    public static void main(String[] args) {
        BankAccount account = new BankAccount("ACC001", "John Kamau", 10000);

        // Cannot access private variables directly
        // account.balance = 50000; // ERROR - balance is private

        // Must use public methods
        account.displayAccountInfo();
        account.deposit(5000);
        account.withdraw(3000);
    }
}

```

```
        // Using getter  
        System.out.println("Current balance: KSh " + account.getBalance());  
    }  
}
```

5.4 Benefits of Encapsulation

- Data protection - prevents unauthorized access
- Data validation - control what values are assigned
- Flexibility - can change internal implementation without affecting other code
- Read-only or write-only access - can provide getters without setters
- Better maintenance - changes are localized to the class

MODULE 6: INHERITANCE - SINGLE INHERITANCE

6.1 What is Inheritance?

Inheritance allows a class to acquire properties and methods of another class.

Terminology:

- Parent Class (Super Class/Base Class) - class being inherited from
- Child Class (Sub Class/Derived Class) - class that inherits
- Keyword: extends

6.2 Single Inheritance Example

```
// Parent Class
public class Person {
    // Attributes
    protected String name;
    protected int age;
    protected String address;

    // Constructor
    public Person(String name, int age, String address) {
        this.name = name;
        this.age = age;
        this.address = address;
    }

    // Methods
    public void displayInfo() {
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
        System.out.println("Address: " + address);
    }

    public void celebrate() {
        System.out.println(name + " is celebrating!");
    }
}

// Child Class
public class Student extends Person {
    // Additional attributes specific to Student
    private String studentId;
    private String course;
    private double gpa;

    // Constructor
    public Student(String name, int age, String address, String studentId,
String course) {
```

```

        // Call parent constructor using super()
        super(name, age, address);
        this.studentId = studentId;
        this.course = course;
        this.gpa = 0.0;
    }

    // Additional methods specific to Student
    public void study() {
        System.out.println(name + " is studying " + course);
    }

    public void setGpa(double gpa) {
        this.gpa = gpa;
    }

    // Override displayInfo to include student-specific details
    @Override
    public void displayInfo() {
        super.displayInfo(); // Call parent method
        System.out.println("Student ID: " + studentId);
        System.out.println("Course: " + course);
        System.out.println("GPA: " + gpa);
    }
}

// Usage
public class Main {
    public static void main(String[] args) {
        // Creating Student object
        Student student = new Student("Alice Wanjiru", 20, "Nairobi", "S001",
"Computer Science");

        student.setGpa(3.7);

        // Student can use both inherited and own methods
        student.displayInfo(); // Overridden method
        student.celebrate();   // Inherited from Person
        student.study();        // Student's own method
    }
}

```

6.3 Another Inheritance Example

```

// Parent Class
public class Employee {
    protected String employeeId;
    protected String name;
    protected double salary;

    public Employee(String employeeId, String name, double salary) {
        this.employeeId = employeeId;
        this.name = name;
        this.salary = salary;
    }
}

```

```

    public void work() {
        System.out.println(name + " is working");
    }

    public double calculateMonthlySalary() {
        return salary / 12;
    }

    public void displayInfo() {
        System.out.println("ID: " + employeeId);
        System.out.println("Name: " + name);
        System.out.println("Annual Salary: KSh " + salary);
    }
}

// Child Class
public class Manager extends Employee {
    private String department;
    private double bonus;

    public Manager(String employeeId, String name, double salary, String
department) {
        super(employeeId, name, salary);
        this.department = department;
        this.bonus = 0.0;
    }

    public void setBonus(double bonus) {
        this.bonus = bonus;
    }

    public void conductMeeting() {
        System.out.println(name + " is conducting a meeting in " + department);
    }

    @Override
    public double calculateMonthlySalary() {
        return (salary + bonus) / 12;
    }

    @Override
    public void displayInfo() {
        super.displayInfo();
        System.out.println("Department: " + department);
        System.out.println("Bonus: KSh " + bonus);
        System.out.println("Monthly Salary: KSh " + calculateMonthlySalary());
    }
}

// Usage
public class Main {
    public static void main(String[] args) {
        Manager manager = new Manager("M001", "John Omondi", 960000, "IT");
        manager.setBonus(120000);
    }
}

```

```
        manager.displayInfo();  
        manager.work();  
        manager.conductMeeting();  
    }  
}
```

MODULE 7: INHERITANCE - MULTILEVEL AND HIERARCHICAL

7.1 Multilevel Inheritance

A class inherits from a child class, creating a chain.

Example: GrandParent → Parent → Child

```
// Level 1: Base Class
public class Animal {
    protected String name;

    public Animal(String name) {
        this.name = name;
    }

    public void eat() {
        System.out.println(name + " is eating");
    }

    public void sleep() {
        System.out.println(name + " is sleeping");
    }
}

// Level 2: Intermediate Class
public class Mammal extends Animal {
    protected int legs;

    public Mammal(String name, int legs) {
        super(name);
        this.legs = legs;
    }

    public void walk() {
        System.out.println(name + " is walking on " + legs + " legs");
    }
}

// Level 3: Final Class
public class Dog extends Mammal {
    private String breed;

    public Dog(String name, String breed) {
        super(name, 4); // Dogs have 4 legs
        this.breed = breed;
    }

    public void bark() {
        System.out.println(name + " (" + breed + ") is barking: Woof! Woof!");
    }

    public void displayInfo() {
        System.out.println("Dog Name: " + name);
    }
}
```

```

        System.out.println("Breed: " + breed);
        System.out.println("Legs: " + legs);
    }
}

// Usage
public class Main {
    public static void main(String[] args) {
        Dog myDog = new Dog("Max", "German Shepherd");

        // Can use methods from all levels
        myDog.displayInfo(); // Dog's method
        myDog.eat();         // Animal's method (Level 1)
        myDog.walk();        // Mammal's method (Level 2)
        myDog.bark();        // Dog's method (Level 3)
    }
}

```

7.2 Hierarchical Inheritance

Multiple classes inherit from a single parent class.

Example: Parent → Child1, Parent → Child2, Parent → Child3

```

// Parent Class
public class Shape {
    protected String color;

    public Shape(String color) {
        this.color = color;
    }

    public void displayColor() {
        System.out.println("Color: " + color);
    }
}

// Child Class 1
public class Circle extends Shape {
    private double radius;

    public Circle(String color, double radius) {
        super(color);
        this.radius = radius;
    }

    public double calculateArea() {
        return Math.PI * radius * radius;
    }

    public void displayInfo() {
        displayColor();
        System.out.println("Shape: Circle");
        System.out.println("Radius: " + radius);
        System.out.println("Area: " + calculateArea());
    }
}

```

```

    }
}

// Child Class 2
public class Rectangle extends Shape {
    private double length;
    private double width;

    public Rectangle(String color, double length, double width) {
        super(color);
        this.length = length;
        this.width = width;
    }

    public double calculateArea() {
        return length * width;
    }

    public void displayInfo() {
        displayColor();
        System.out.println("Shape: Rectangle");
        System.out.println("Length: " + length);
        System.out.println("Width: " + width);
        System.out.println("Area: " + calculateArea());
    }
}

// Child Class 3
public class Triangle extends Shape {
    private double base;
    private double height;

    public Triangle(String color, double base, double height) {
        super(color);
        this.base = base;
        this.height = height;
    }

    public double calculateArea() {
        return 0.5 * base * height;
    }

    public void displayInfo() {
        displayColor();
        System.out.println("Shape: Triangle");
        System.out.println("Base: " + base);
        System.out.println("Height: " + height);
        System.out.println("Area: " + calculateArea());
    }
}

// Usage
public class Main {
    public static void main(String[] args) {
        Circle circle = new Circle("Red", 5.0);
    }
}

```

```
Rectangle rectangle = new Rectangle("Blue", 4.0, 6.0);
Triangle triangle = new Triangle("Green", 3.0, 4.0);

System.out.println("=== CIRCLE ===");
circle.displayInfo();

System.out.println("\n=== RECTANGLE ===");
rectangle.displayInfo();

System.out.println("\n=== TRIANGLE ===");
triangle.displayInfo();
    }
}
```

MODULE 8: METHOD OVERRIDING AND SUPER KEYWORD

8.1 Method Overriding

Method overriding occurs when a child class provides its own implementation of a method already defined in the parent class.

Rules:

- Method must have same name as in parent class
- Method must have same parameters
- Method must have same or compatible return type
- Use `@Override` annotation (optional but recommended)

```
public class Vehicle {
    protected String brand;
    protected int speed;

    public Vehicle(String brand) {
        this.brand = brand;
        this.speed = 0;
    }

    public void start() {
        System.out.println(brand + " is starting");
    }

    public void accelerate() {
        speed += 10;
        System.out.println(brand + " speed: " + speed + " km/h");
    }
}

public class Car extends Vehicle {
    private int doors;

    public Car(String brand, int doors) {
        super(brand);
        this.doors = doors;
    }

    // Method Overriding
    @Override
    public void start() {
        System.out.println(brand + " car is starting with key ignition");
    }

    @Override
```

```

        public void accelerate() {
            speed += 20; // Cars accelerate faster
            System.out.println(brand + " car speed: " + speed + " km/h");
        }
    }

    public class Motorcycle extends Vehicle {

        public Motorcycle(String brand) {
            super(brand);
        }

        // Method Overriding
        @Override
        public void start() {
            System.out.println(brand + " motorcycle is starting with kick start");
        }

        @Override
        public void accelerate() {
            speed += 30; // Motorcycles accelerate even faster
            System.out.println(brand + " motorcycle speed: " + speed + " km/h");
        }
    }

    // Usage
    public class Main {
        public static void main(String[] args) {
            Car car = new Car("Toyota", 4);
            Motorcycle bike = new Motorcycle("Yamaha");

            car.start();
            car.accelerate();
            car.accelerate();

            System.out.println();

            bike.start();
            bike.accelerate();
            bike.accelerate();
        }
    }
}

```

8.2 Super Keyword

The super keyword is used to:

- Call parent class constructor: `super()`
- Call parent class method: `super.methodName()`
- Access parent class variable: `super.variableName`

MODULE 9: POLYMORPHISM

9.1 What is Polymorphism?

Polymorphism means "many forms" - same method behaves differently in different contexts.

Types:

1. Compile-time Polymorphism (Method Overloading)
2. Runtime Polymorphism (Method Overriding)

9.2 Runtime Polymorphism Example

```
// Parent class
public class Animal {
    public void makeSound() {
        System.out.println("Animal makes a sound");
    }

    public void eat() {
        System.out.println("Animal is eating");
    }
}

// Child class 1
public class Dog extends Animal {
    @Override
    public void makeSound() {
        System.out.println("Dog barks: Woof! Woof!");
    }
}

// Child class 2
public class Cat extends Animal {
    @Override
    public void makeSound() {
        System.out.println("Cat meows: Meow! Meow!");
    }
}

// Child class 3
public class Cow extends Animal {
    @Override
    public void makeSound() {
        System.out.println("Cow moos: Moo! Moo!");
    }
}

// Demonstrating Polymorphism
public class Main {
```

```

public static void main(String[] args) {
    // Parent reference, child object - THIS IS POLYMORPHISM
    Animal animal1 = new Dog();
    Animal animal2 = new Cat();
    Animal animal3 = new Cow();

    // Same method call, different behavior
    animal1.makeSound(); // Dog's version
    animal2.makeSound(); // Cat's version
    animal3.makeSound(); // Cow's version

    // Array of parent type holding different child objects
    Animal[] animals = {new Dog(), new Cat(), new Cow()};

    System.out.println("\nAll animals making sounds:");
    for (Animal animal : animals) {
        animal.makeSound();
    }
}

```

9.3 Practical Polymorphism Example - Payment System

```

public class Payment {
    protected double amount;

    public Payment(double amount) {
        this.amount = amount;
    }

    public void processPayment() {
        System.out.println("Processing payment of KSh " + amount);
    }
}

public class CreditCardPayment extends Payment {
    private String cardNumber;

    public CreditCardPayment(double amount, String cardNumber) {
        super(amount);
        this.cardNumber = cardNumber;
    }

    @Override
    public void processPayment() {
        System.out.println("Processing Credit Card payment of KSh " + amount);
        System.out.println("Card ending in: " +
            cardNumber.substring(cardNumber.length() - 4));
    }
}

public class MpesaPayment extends Payment {
    private String phoneNumber;

    public MpesaPayment(double amount, String phoneNumber) {

```

```

        super(amount);
        this.phoneNumber = phoneNumber;
    }

    @Override
    public void processPayment() {
        System.out.println("Processing M-Pesa payment of KSh " + amount);
        System.out.println("Phone: " + phoneNumber);
        System.out.println("Check your phone for M-Pesa prompt");
    }
}

public class CashPayment extends Payment {

    public CashPayment(double amount) {
        super(amount);
    }

    @Override
    public void processPayment() {
        System.out.println("Processing Cash payment of KSh " + amount);
        System.out.println("Please pay at the counter");
    }
}

// Usage
public class Main {
    public static void main(String[] args) {
        // Polymorphism in action
        Payment payment1 = new CreditCardPayment(5000, "1234567890123456");
        Payment payment2 = new MpesaPayment(3000, "07000000000");
        Payment payment3 = new CashPayment(2000);

        // Same method call, different implementations
        payment1.processPayment();
        System.out.println();
        payment2.processPayment();
        System.out.println();
        payment3.processPayment();
    }
}

```

MODULE 10: ABSTRACT CLASSES

10.1 What is an Abstract Class?

An abstract class is a class that:

- Cannot be instantiated (cannot create objects)
- May contain abstract methods (methods without body)
- May contain concrete methods (regular methods)
- Used as a blueprint for other classes
- Keyword: abstract

10.2 Abstract Class Example

```
// Abstract class
public abstract class Employee {
    protected String name;
    protected String employeeId;
    protected double baseSalary;

    public Employee(String name, String employeeId, double baseSalary) {
        this.name = name;
        this.employeeId = employeeId;
        this.baseSalary = baseSalary;
    }

    // Abstract method - no implementation
    public abstract double calculateSalary();

    // Abstract method
    public abstract void displayDetails();

    // Concrete method - has implementation
    public void work() {
        System.out.println(name + " is working");
    }

    public String getName() {
        return name;
    }
}

// Concrete class implementing abstract class
public class FullTimeEmployee extends Employee {
    private double bonus;

    public FullTimeEmployee(String name, String employeeId, double baseSalary,
double bonus) {
        super(name, employeeId, baseSalary);
        this.bonus = bonus;
    }

    // Must implement all abstract methods
```

```

@Override
public double calculateSalary() {
    return baseSalary + bonus;
}

@Override
public void displayDetails() {
    System.out.println("Full-Time Employee");
    System.out.println("Name: " + name);
    System.out.println("ID: " + employeeId);
    System.out.println("Base Salary: KSh " + baseSalary);
    System.out.println("Bonus: KSh " + bonus);
    System.out.println("Total Salary: KSh " + calculateSalary());
}
}

public class PartTimeEmployee extends Employee {
    private int hoursWorked;
    private double hourlyRate;

    public PartTimeEmployee(String name, String employeeId, int hoursWorked,
double hourlyRate) {
        super(name, employeeId, 0);
        this.hoursWorked = hoursWorked;
        this.hourlyRate = hourlyRate;
    }

    @Override
    public double calculateSalary() {
        return hoursWorked * hourlyRate;
    }

    @Override
    public void displayDetails() {
        System.out.println("Part-Time Employee");
        System.out.println("Name: " + name);
        System.out.println("ID: " + employeeId);
        System.out.println("Hours Worked: " + hoursWorked);
        System.out.println("Hourly Rate: KSh " + hourlyRate);
        System.out.println("Total Salary: KSh " + calculateSalary());
    }
}

// Usage
public class Main {
    public static void main(String[] args) {
        // Cannot create object of abstract class
        // Employee emp = new Employee(); // ERROR!

        // Create objects of concrete classes
        Employee emp1 = new FullTimeEmployee("John Doe", "FT001", 50000,
10000);
        Employee emp2 = new PartTimeEmployee("Jane Smith", "PT001", 80, 500);

        emp1.displayDetails();
    }
}

```

```
        System.out.println();  
        emp2.displayDetails();  
    }  
}
```

MODULE 11: INTERFACES

11.1 What is an Interface?

An interface is a completely abstract class that:

- Contains only abstract methods (before Java 8)
- All methods are public and abstract by default
- All variables are public, static, and final
- A class implements an interface using "implements" keyword
- A class can implement multiple interfaces
- Keyword: interface

11.2 Interface Example

```
// Interface
public interface Printable {
    // Abstract methods (public and abstract by default)
    void print();
    void preview();
}

// Class implementing interface
public class Document implements Printable {
    private String content;

    public Document(String content) {
        this.content = content;
    }

    // Must implement all interface methods
    @Override
    public void print() {
        System.out.println("Printing document: " + content);
    }

    @Override
    public void preview() {
        System.out.println("Previewing document: " + content);
    }
}

public class Invoice implements Printable {
    private String invoiceNumber;
    private double amount;

    public Invoice(String invoiceNumber, double amount) {
        this.invoiceNumber = invoiceNumber;
        this.amount = amount;
    }

    @Override
    public void print() {
```

```

        System.out.println("Printing Invoice #" + invoiceNumber);
        System.out.println("Amount: KSh " + amount);
    }

    @Override
    public void preview() {
        System.out.println("Previewing Invoice #" + invoiceNumber);
    }
}

// Usage
public class Main {
    public static void main(String[] args) {
        Printable doc = new Document("Annual Report 2024");
        Printable inv = new Invoice("INV-001", 15000);

        doc.preview();
        doc.print();

        System.out.println();

        inv.preview();
        inv.print();
    }
}

```

11.3 Multiple Interface Implementation

```

// Interface 1
public interface Flyable {
    void fly();
}

// Interface 2
public interface Swimmable {
    void swim();
}

// Class implementing multiple interfaces
public class Duck implements Flyable, Swimmable {
    private String name;

    public Duck(String name) {
        this.name = name;
    }

    @Override
    public void fly() {
        System.out.println(name + " is flying");
    }

    @Override
    public void swim() {
        System.out.println(name + " is swimming");
    }
}

```

```
}

// Usage
public class Main {
    public static void main(String[] args) {
        Duck duck = new Duck("Donald");
        duck.fly();
        duck.swim();
    }
}
```

11.4 Abstract Class vs Interface

Abstract Class:

- Can have both abstract and concrete methods
- Can have instance variables
- Can have constructors
- Use "extends" keyword (single inheritance)

Interface:

- All methods are abstract (before Java 8)
- Only constants (public static final)
- No constructors
- Use "implements" keyword (multiple interfaces)

MODULE 12: EXCEPTION HANDLING

12.1 What is an Exception?

An exception is an error that occurs during program execution.

Common Exceptions:

- `ArithmeticException` - Division by zero
- `NullPointerException` - Using null reference
- `ArrayIndexOutOfBoundsException` - Invalid array index
- `NumberFormatException` - Invalid number format
- `FileNotFoundException` - File not found

12.2 Try-Catch Block

```
public class ExceptionExample {
    public static void main(String[] args) {

        // Example 1: Handling ArithmeticException
        try {
            int result = 10 / 0; // Will throw ArithmeticException
            System.out.println("Result: " + result);
        } catch (ArithmeticException e) {
            System.out.println("Error: Cannot divide by zero!");
            System.out.println("Exception message: " + e.getMessage());
        }

        System.out.println("Program continues...");

        // Example 2: Handling ArrayIndexOutOfBoundsException
        try {
            int[] numbers = {1, 2, 3};
            System.out.println(numbers[5]); // Invalid index
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Error: Array index out of bounds!");
        }

        // Example 3: Multiple catch blocks
        try {
            String text = null;
            System.out.println(text.length()); // NullPointerException
        } catch (NullPointerException e) {
            System.out.println("Error: Null reference!");
        } catch (Exception e) {
            System.out.println("Error: Something went wrong!");
        }
    }
}
```

12.3 Try-Catch-Finally Block

```
import java.util.Scanner;

public class BankAccount {
    private double balance;

    public BankAccount(double initialBalance) {
        this.balance = initialBalance;
    }

    public void withdraw(double amount) {
        try {
            if (amount <= 0) {
                throw new IllegalArgumentException("Amount
must be positive");
            }
            if (amount > balance) {
                throw new
IllegalArgumentException("Insufficient funds");
            }

            balance -= amount;
            System.out.println("Withdrawal successful: KSh
" + amount);
            System.out.println("New balance: KSh " +
balance);
        } catch (IllegalArgumentException e) {
            System.out.println("Error: " + e.getMessage());
        } finally {
            System.out.println("Transaction completed");
        }
    }

    public static void main(String[] args) {
        BankAccount account = new BankAccount(10000);

        account.withdraw(5000);    // Success
        System.out.println();
        account.withdraw(-100);    // Error: negative amount
        System.out.println();
    }
}
```

```

        account.withdraw(20000); // Error: insufficient
funds
    }
}

```

12.4 Custom Exceptions

```

// Custom Exception Class
public class InvalidAgeException extends Exception {
    public InvalidAgeException(String message) {
        super(message);
    }
}

public class Student {
    private String name;
    private int age;

    public void setAge(int age) throws InvalidAgeException {
        if (age < 0 || age > 100) {
            throw new InvalidAgeException("Age must be between 0 and 100");
        }
        this.age = age;
    }

    public static void main(String[] args) {
        Student student = new Student();

        try {
            student.setAge(150); // Will throw custom exception
        } catch (InvalidAgeException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}

```

MODULE 13: COLLECTIONS FRAMEWORK - ARRAYLIST

13.1 What is ArrayList?

ArrayList is a resizable array that:

- Can grow and shrink dynamically
- Stores objects (not primitives directly)
- Provides methods for adding, removing, searching
- Part of java.util package
- More flexible than regular arrays

13.2 ArrayList Basic Operations

```
import java.util.ArrayList;

public class ArrayListExample {
    public static void main(String[] args) {

        // Creating ArrayList
        ArrayList<String> students = new ArrayList<>();

        // Adding elements
        students.add("John Kamau");
        students.add("Mary Wanjiru");
        students.add("Peter Ochieng");
        students.add("Grace Akinyi");

        // Display ArrayList
        System.out.println("Students: " + students);
        System.out.println("Total students: " + students.size());

        // Accessing elements
        System.out.println("First student: " + students.get(0));
        System.out.println("Third student: " + students.get(2));

        // Modifying element
        students.set(1, "Mary Njeri");
        System.out.println("After modification: " + students);

        // Checking if element exists
        if (students.contains("Peter Ochieng")) {
            System.out.println("Peter is in the list");
        }

        // Removing element
        students.remove("Grace Akinyi");
        students.remove(0); // Remove by index
        System.out.println("After removal: " + students);

        // Iterating through ArrayList
        System.out.println("\nAll students:");
        for (String student : students) {
```

```

        System.out.println("- " + student);
    }

    // Clear all elements
    students.clear();
    System.out.println("After clear: " + students);
    System.out.println("Is empty? " + students.isEmpty());
}
}

```

13.3 ArrayList with Custom Objects

```

import java.util.ArrayList;

class Student {
    private String name;
    private int age;
    private double gpa;

    public Student(String name, int age, double gpa) {
        this.name = name;
        this.age = age;
        this.gpa = gpa;
    }

    public void displayInfo() {
        System.out.println("Name: " + name + ", Age: " + age + ", GPA: " +
gpa);
    }

    public double getGpa() {
        return gpa;
    }

    public String getName() {
        return name;
    }
}

public class StudentManagement {
    public static void main(String[] args) {

        ArrayList<Student> students = new ArrayList<>();

        // Adding students
        students.add(new Student("Alice Muthoni", 20, 3.8));
        students.add(new Student("Bob Kipchoge", 21, 3.5));
        students.add(new Student("Carol Wambui", 19, 3.9));
        students.add(new Student("David Omondi", 22, 3.2));

        // Display all students
        System.out.println("All Students:");
        for (Student student : students) {
            student.displayInfo();
        }
    }
}

```

```
// Find students with GPA > 3.5
System.out.println("\nHonors Students (GPA > 3.5):");
for (Student student : students) {
    if (student.getGpa() > 3.5) {
        student.displayInfo();
    }
}

// Count total students
System.out.println("\nTotal students: " + students.size());
}
}
```

MODULE 14: COLLECTIONS FRAMEWORK - HASHMAP

14.1 What is HashMap?

HashMap stores data in key-value pairs:

- Each key must be unique
- Keys map to values
- Fast retrieval by key
- Part of java.util package
- Useful for dictionaries, phonebooks, etc.

14.2 HashMap Basic Operations

```
import java.util.HashMap;

public class HashMapExample {
    public static void main(String[] args) {

        // Creating HashMap (Key: String, Value: Integer)
        HashMap<String, Integer> ages = new HashMap<>();

        // Adding key-value pairs
        ages.put("John", 25);
        ages.put("Mary", 30);
        ages.put("Peter", 28);
        ages.put("Grace", 22);

        // Display HashMap
        System.out.println("Ages: " + ages);

        // Getting value by key
        System.out.println("John's age: " + ages.get("John"));
        System.out.println("Mary's age: " + ages.get("Mary"));

        // Check if key exists
        if (ages.containsKey("Peter")) {
            System.out.println("Peter's age is in the map");
        }

        // Update value
        ages.put("John", 26); // Updates existing key
        System.out.println("Updated ages: " + ages);

        // Remove entry
        ages.remove("Grace");
        System.out.println("After removal: " + ages);

        // Iterating through HashMap
        System.out.println("\nAll entries:");
        for (String name : ages.keySet()) {
            System.out.println(name + " is " + ages.get(name) + " years old");
        }
    }
}
```

```

        // Size
        System.out.println("\nTotal entries: " + ages.size());
    }
}

```

14.3 Practical HashMap Example - Student Grades

```

import java.util.HashMap;
import java.util.Scanner;

public class GradeBook {
    private HashMap<String, Double> grades;

    public GradeBook() {
        grades = new HashMap<>();
    }

    public void addStudent(String name, double grade) {
        grades.put(name, grade);
        System.out.println("Added: " + name + " - " + grade);
    }

    public void displayAllGrades() {
        System.out.println("\n=== ALL STUDENT GRADES ===");
        for (String name : grades.keySet()) {
            double grade = grades.get(name);
            System.out.println(name + ": " + grade + " (" +
getLetterGrade(grade) + ")");
        }
    }

    public String getLetterGrade(double grade) {
        if (grade >= 90) return "A";
        else if (grade >= 80) return "B";
        else if (grade >= 70) return "C";
        else if (grade >= 60) return "D";
        else return "F";
    }

    public double getAverageGrade() {
        if (grades.isEmpty()) return 0;

        double sum = 0;
        for (double grade : grades.values()) {
            sum += grade;
        }
        return sum / grades.size();
    }

    public static void main(String[] args) {
        GradeBook gradeBook = new GradeBook();

        gradeBook.addStudent("Alice Muthoni", 85.5);
        gradeBook.addStudent("Bob Kipchoge", 92.0);
    }
}

```

```
gradeBook.addStudent("Carol Wambui", 78.5);
gradeBook.addStudent("David Omondi", 88.0);

gradeBook.displayAllGrades();

System.out.println("\nClass Average: " + gradeBook.getAverageGrade());
}
}
```

MODULE 15: FILE I/O OPERATIONS

15.1 Writing to a File

```
import java.io.FileWriter;
import java.io.IOException;

public class FileWriteExample {
    public static void main(String[] args) {

        try {
            FileWriter writer = new FileWriter("student_data.txt");

            writer.write("Student Records\n");
            writer.write("=====\n");
            writer.write("Name: John Kamau\n");
            writer.write("Age: 20\n");
            writer.write("Course: Computer Science\n");
            writer.write("GPA: 3.7\n");

            writer.close();
            System.out.println("Data written to file successfully!");

        } catch (IOException e) {
            System.out.println("Error writing to file: " + e.getMessage());
        }
    }
}
```

15.2 Reading from a File

```
import java.io.File;
import java.io.FileNotFoundException;
import java.util.Scanner;

public class FileReadExample {
    public static void main(String[] args) {

        try {
            File file = new File("student_data.txt");
            Scanner reader = new Scanner(file);

            System.out.println("Reading from file:");
            System.out.println("=====");

            while (reader.hasNextLine()) {
                String line = reader.nextLine();
                System.out.println(line);
            }

            reader.close();

        } catch (FileNotFoundException e) {
            System.out.println("File not found: " + e.getMessage());
        }
    }
}
```

```
    }  
}
```

15.3 Complete File Operations Example

```
import java.io.*;  
import java.util.Scanner;  
  
public class StudentFileManager {  
    private static final String FILENAME = "students.txt";  
  
    public static void saveStudent(String name, int age, double gpa) {  
        try {  
            FileWriter writer = new FileWriter(FILENAME, true); // true =  
append mode  
            writer.write(name + "," + age + "," + gpa + "\n");  
            writer.close();  
            System.out.println("Student saved successfully!");  
        } catch (IOException e) {  
            System.out.println("Error saving student: " + e.getMessage());  
        }  
    }  
  
    public static void displayAllStudents() {  
        try {  
            File file = new File(FILENAME);  
            Scanner reader = new Scanner(file);  
  
            System.out.println("\n=== ALL STUDENTS ===");  
            while (reader.hasNextLine()) {  
                String line = reader.nextLine();  
                String[] parts = line.split(",");  
                System.out.println("Name: " + parts[0] +  
                                ", Age: " + parts[1] +  
                                ", GPA: " + parts[2]);  
            }  
            reader.close();  
        } catch (FileNotFoundException e) {  
            System.out.println("No student records found");  
        }  
    }  
  
    public static void main(String[] args) {  
        saveStudent("Alice Muthoni", 20, 3.8);  
        saveStudent("Bob Kipchoge", 21, 3.5);  
        saveStudent("Carol Wambui", 19, 3.9);  
  
        displayAllStudents();  
    }  
}
```

MODULE 16: COMPLETE PROJECT - STUDENT MANAGEMENT SYSTEM

16.1 Project Overview

A complete system to manage students with:

- Add new students
- Display all students
- Search for students
- Update student information
- Delete students
- Calculate average GPA
- Save to file and load from file

16.2 Complete Code

```
import java.util.ArrayList;
import java.util.Scanner;
import java.io.*;

// Student Class
class Student {
    private String studentId;
    private String name;
    private int age;
    private String course;
    private double gpa;

    public Student(String studentId, String name, int age, String course, double gpa) {
        this.studentId = studentId;
        this.name = name;
        this.age = age;
        this.course = course;
        this.gpa = gpa;
    }

    // Getters
    public String getStudentId() { return studentId; }
    public String getName() { return name; }
    public int getAge() { return age; }
    public String getCourse() { return course; }
    public double getGpa() { return gpa; }

    // Setters
    public void setName(String name) { this.name = name; }
    public void setAge(int age) { this.age = age; }
    public void setCourse(String course) { this.course = course; }
    public void setGpa(double gpa) { this.gpa = gpa; }

    public void displayInfo() {
        System.out.println("ID: " + studentId);
        System.out.println("Name: " + name);
        System.out.println("Age: " + age);
        System.out.println("Course: " + course);
        System.out.println("GPA: " + gpa);
        System.out.println("-----");
    }
}
```

```

        public String toFileString() {
            return studentId + "," + name + "," + age + "," + course + "," + gpa;
        }
    }

// Student Management System Class
class StudentManagementSystem {
    private ArrayList<Student> students;
    private Scanner scanner;
    private static final String FILENAME = "students.txt";

    public StudentManagementSystem() {
        students = new ArrayList<>();
        scanner = new Scanner(System.in);
        loadFromFile();
    }

    public void addStudent() {
        System.out.println("\n=== ADD NEW STUDENT ===");
        System.out.print("Enter Student ID: ");
        String id = scanner.nextLine();

        // Check if ID already exists
        for (Student s : students) {
            if (s.getId().equals(id)) {
                System.out.println("Error: Student ID already exists!");
                return;
            }
        }

        System.out.print("Enter Name: ");
        String name = scanner.nextLine();
        System.out.print("Enter Age: ");
        int age = scanner.nextInt();
        scanner.nextLine(); // Consume newline
        System.out.print("Enter Course: ");
        String course = scanner.nextLine();
        System.out.print("Enter GPA: ");
        double gpa = scanner.nextDouble();
        scanner.nextLine();

        Student student = new Student(id, name, age, course, gpa);
        students.add(student);
        System.out.println("Student added successfully!");
        saveToFile();
    }

    public void displayAllStudents() {
        System.out.println("\n=== ALL STUDENTS ===");
        if (students.isEmpty()) {
            System.out.println("No students found.");
            return;
        }

        for (Student student : students) {
            student.displayInfo();
        }
        System.out.println("Total students: " + students.size());
    }

    public void searchStudent() {

```

```

        System.out.print("\nEnter Student ID to search: ");
        String id = scanner.nextLine();

        for (Student student : students) {
            if (student.getStudentId().equals(id)) {
                System.out.println("\nStudent Found:");
                student.displayInfo();
                return;
            }
        }
        System.out.println("Student not found!");
    }

    public void updateStudent() {
        System.out.print("\nEnter Student ID to update: ");
        String id = scanner.nextLine();

        for (Student student : students) {
            if (student.getStudentId().equals(id)) {
                System.out.println("\nCurrent Information:");
                student.displayInfo();

                System.out.print("Enter new Name (or press Enter to skip): ");
                String name = scanner.nextLine();
                if (!name.isEmpty()) student.setName(name);

                System.out.print("Enter new Age (or 0 to skip): ");
                int age = scanner.nextInt();
                scanner.nextLine();
                if (age != 0) student.setAge(age);

                System.out.print("Enter new Course (or press Enter to skip): ");
                String course = scanner.nextLine();
                if (!course.isEmpty()) student.setCourse(course);

                System.out.print("Enter new GPA (or 0 to skip): ");
                double gpa = scanner.nextDouble();
                scanner.nextLine();
                if (gpa != 0) student.setGpa(gpa);

                System.out.println("Student updated successfully!");
                saveToFile();
                return;
            }
        }
        System.out.println("Student not found!");
    }

    public void deleteStudent() {
        System.out.print("\nEnter Student ID to delete: ");
        String id = scanner.nextLine();

        for (int i = 0; i < students.size(); i++) {
            if (students.get(i).getStudentId().equals(id)) {
                students.remove(i);
                System.out.println("Student deleted successfully!");
                saveToFile();
                return;
            }
        }
        System.out.println("Student not found!");
    }
}

```

```

public void calculateAverageGPA() {
    if (students.isEmpty()) {
        System.out.println("No students to calculate average.");
        return;
    }

    double sum = 0;
    for (Student student : students) {
        sum += student.getGpa();
    }
    double average = sum / students.size();
    System.out.println("\nAverage GPA: " + String.format("%.2f", average));
}

private void saveToFile() {
    try {
        FileWriter writer = new FileWriter(FILENAME);
        for (Student student : students) {
            writer.write(student.toFileString() + "\n");
        }
        writer.close();
    } catch (IOException e) {
        System.out.println("Error saving to file: " + e.getMessage());
    }
}

private void loadFromFile() {
    try {
        File file = new File(FILENAME);
        if (!file.exists()) return;

        Scanner reader = new Scanner(file);
        while (reader.hasNextLine()) {
            String line = reader.nextLine();
            String[] parts = line.split(",");
            Student student = new Student(parts[0], parts[1],
                Integer.parseInt(parts[2]), parts[3],
                Double.parseDouble(parts[4]));
            students.add(student);
        }
        reader.close();
        System.out.println("Loaded " + students.size() + " students from file.");
    } catch (FileNotFoundException e) {
        // File doesn't exist yet, that's okay
    }
}

// Main Class
public class Main {
    public static void main(String[] args) {
        StudentManagementSystem sms = new StudentManagementSystem();
        Scanner scanner = new Scanner(System.in);

        while (true) {
            System.out.println("\n=== STUDENT MANAGEMENT SYSTEM ===");
            System.out.println("1. Add Student");
            System.out.println("2. Display All Students");
            System.out.println("3. Search Student");
            System.out.println("4. Update Student");
            System.out.println("5. Delete Student");

```

```
System.out.println("6. Calculate Average GPA");
System.out.println("7. Exit");
System.out.print("Enter choice: ");

int choice = scanner.nextInt();
scanner.nextLine(); // Consume newline

switch (choice) {
    case 1: sms.addStudent(); break;
    case 2: sms.displayAllStudents(); break;
    case 3: sms.searchStudent(); break;
    case 4: sms.updateStudent(); break;
    case 5: sms.deleteStudent(); break;
    case 6: sms.calculateAverageGPA(); break;
    case 7:
        System.out.println("Thank you for using Student Management System!");
        System.exit(0);
    default:
        System.out.println("Invalid choice! Please try again.");
}
}
}
```

MODULE 17: COMPLETE PROJECT - BANK ACCOUNT SYSTEM

17.1 Project Features

Complete banking system with:

- Create account
- Deposit money
- Withdraw money
- Check balance
- Transaction history
- Transfer between accounts

17.2 Simplified Implementation

Due to space constraints, here is the core implementation:

```
import java.util.*;

class BankAccount {
    private String accountNumber;
    private String accountHolder;
    private double balance;
    private ArrayList<String> transactionHistory;

    public BankAccount(String accountNumber, String accountHolder, double initialBalance)
    {
        this.accountNumber = accountNumber;
        this.accountHolder = accountHolder;
        this.balance = initialBalance;
        this.transactionHistory = new ArrayList<>();
        addTransaction("Account created with balance: KSh " + initialBalance);
    }

    public void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
            addTransaction("Deposited: KSh " + amount);
            System.out.println("Deposit successful. New balance: KSh " + balance);
        } else {
            System.out.println("Invalid amount");
        }
    }

    public void withdraw(double amount) {
        if (amount > 0 && amount <= balance) {
            balance -= amount;
            addTransaction("Withdrew: KSh " + amount);
            System.out.println("Withdrawal successful. New balance: KSh " + balance);
        } else if (amount > balance) {
            System.out.println("Insufficient funds");
        } else {
            System.out.println("Invalid amount");
        }
    }
}
```

```

    }
}

public void displayBalance() {
    System.out.println("Account: " + accountNumber);
    System.out.println("Holder: " + accountHolder);
    System.out.println("Balance: KSh " + balance);
}

public void displayTransactionHistory() {
    System.out.println("\n=== TRANSACTION HISTORY ===");
    for (String transaction : transactionHistory) {
        System.out.println(transaction);
    }
}

private void addTransaction(String transaction) {
    String timestamp = new Date().toString();
    transactionHistory.add(timestamp + " - " + transaction);
}

public String getAccountNumber() { return accountNumber; }
public double getBalance() { return balance; }
}

```

MODULE 19: BEST PRACTICES AND DESIGN PATTERNS

19.1 OOP Best Practices

- **Encapsulation** - Always make fields private and provide getters/setters
- **Single Responsibility** - Each class should have one clear purpose
- **DRY Principle** - Don't Repeat Yourself - avoid code duplication
- **Meaningful Names** - Use descriptive names for classes, methods, variables
- **Comments** - Add comments to explain complex logic
- **Error Handling** - Always use try-catch for risky operations
- **Consistent Formatting** - Use consistent indentation and style
- **Testing** - Test your code with different inputs

19.2 Common Mistakes to Avoid

- Not encapsulating data (making everything public)
- Creating God classes (classes that do too much)
- Ignoring exceptions (empty catch blocks)
- Not closing file streams
- Using primitive types when objects are better
- Not using inheritance when it makes sense
- Overusing inheritance when composition is better
- Not validating input data

19.3 Code Quality Checklist

Before submitting your project:

- ✓ All variables properly encapsulated
- ✓ Constructors initialize all fields
- ✓ Methods have clear, single purposes
- ✓ Exception handling in place
- ✓ No unused variables or methods
- ✓ Consistent naming conventions
- ✓ Code properly indented
- ✓ Comments explain complex logic
- ✓ No hardcoded values (use constants)

- ✓ Tested with various inputs
- ✓ File operations close resources
- ✓ No duplicate code

CONGRATULATIONS! 🎉

You have completed the comprehensive Java OOP tutorial!

What You've Learned:

- ✓ OOP fundamentals and four pillars
- ✓ Classes and objects creation
- ✓ Constructors and their overloading
- ✓ Methods and method overloading
- ✓ Encapsulation and data hiding
- ✓ Inheritance (single, multilevel, hierarchical)
- ✓ Method overriding and super keyword
- ✓ Polymorphism (compile-time and runtime)
- ✓ Abstract classes and their usage
- ✓ Interfaces and multiple implementation
- ✓ Exception handling (try-catch-finally)
- ✓ Custom exceptions
- ✓ ArrayList operations
- ✓ HashMap operations
- ✓ File I/O (reading and writing)
- ✓ Complete real-world projects
- ✓ Best practices and design patterns

Practice Projects:

Build these projects to master Java OOP:

1. Library Management System
2. Hospital Management System

3. Inventory Management System
4. Employee Payroll System
5. Online Shopping Cart
6. Quiz Application
7. Contact Management System
8. Expense Tracker
9. Hotel Reservation System
10. Vehicle Rental System

Next Steps:

1. Practice building complete applications
2. Learn Java GUI (Swing/JavaFX)
3. Study design patterns (Singleton, Factory, etc.)
4. Learn JDBC for database connectivity
5. Explore web development with Java (Servlets, JSP)
6. Study Spring Framework
7. Build portfolio projects
8. Contribute to open-source projects
9. Prepare for Java certification
10. Keep coding every day!

Resources:

- **Oracle Java Documentation:** docs.oracle.com/javase
- **Java Tutorials:** docs.oracle.com/javase/tutorial
- **Practice Problems:** codingbat.com/java
- **LeetCode:** leetcode.com
- **HackerRank:** hackerrank.com/domains/java
- **W3Schools Java:** w3schools.com/java
- **Java Community:** stackoverflow.com

**Happy Coding! Master Java OOP and Build Amazing
Applications! ☕ 🚀**